



# PROIECT Paradigme de proiectare a aplicațiilor web

---

## DataInsight Dashboard – Aplicație Web pentru Vizualizare Date

---

## Cuprins:

|                                                                    |   |
|--------------------------------------------------------------------|---|
| Capitolul 1. Introducere .....                                     | 3 |
| 1.1. Contextul și motivația aplicației.....                        | 3 |
| 1.2. Obiectivele proiectului.....                                  | 3 |
| Capitolul 2. Proiectare .....                                      | 3 |
| 2.1. Paradigme Utilizate și Justificare .....                      | 3 |
| 2.1.1. Arhitectura MVC (Model-View-Controller).....                | 3 |
| 2.1.2. API REST (Representational State Transfer).....             | 4 |
| 2.1.3. ORM Code First (Object-Relational Mapping) .....            | 4 |
| 2.1.4. Service Layer Pattern .....                                 | 4 |
| 2.2. Arhitectura Aplicației .....                                  | 4 |
| 2.4. Proiectarea Bazei de Date.....                                | 4 |
| Capitolul 3. Implementare .....                                    | 5 |
| 3.1. Business Layer (Nivelul Servicii) .....                       | 5 |
| 3.1.1. Serviciul de Planuri (PlanService).....                     | 5 |
| 3.1.2. Serviciul de Fișiere (FileService) .....                    | 5 |
| 3.1.3. Serviciul de Utilizatori (UserService) - Admin vs User..... | 6 |
| 3.2. Librării Suplimentare Utilizate .....                         | 6 |
| 3.3. Librării suplimentare utilizate .....                         | 6 |
| Capitolul 4. Utilizare.....                                        | 6 |
| 4.1. Pașii de Instalare .....                                      | 6 |
| 4.2. Mod de Utilizare.....                                         | 7 |
| 4.3.1. Ghid Utilizator .....                                       | 7 |
| 4.3.2. Ghid Administrator .....                                    | 7 |

# Capitolul 1. Introducere

## 1.1. Contextul și motivația aplicației

În era digitală actuală, datele reprezintă una dintre cele mai valoroase resurse pentru companii și indivizi. Totuși, datele brute (sub formă de fișiere CSV sau Excel) sunt greu de interpretat fără instrumente adecvate de vizualizare. DataInsight Dashboard a fost creată pentru a răspunde acestei nevoi, oferind o platformă intuitivă prin care utilizatorii pot transforma seturi de date statice în grafice dinamice și interactive, fără a necesita cunoștințe de programare.

## 1.2. Obiectivele proiectului

Principalul obiectiv al acestui proiect este dezvoltarea unei aplicații web robuste, scalabile și sigure, care să respecte bunele practici și paradigmele moderne de proiectare web. Obiective specifice:

- Implementarea unei arhitecturi decuplate (Frontend vs Backend).
- Gestiunea eficientă a utilizatorilor și a permisiunilor (Role-Based Access Control).
- Monetizarea serviciului prin planuri de abonament cu limitări specifice (Free, Pro, Enterprise).
- Asigurarea integrității datelor prin mecanisme de Soft Delete.

# Capitolul 2. Proiectare

## 2.1. Paradigme Utilizate și Justificare

Arhitectura software stă la baza oricărei aplicații de succes. Pentru **DataInsight Dashboard**, am ales un set de paradigme care favorizează mentenabilitatea și extensibilitatea codului.

### 2.1.1. Arhitectura MVC (Model-View-Controller)

Deși aplicația folosește un Frontend separat (React), Backend-ul (API-ul) este structurat intern pe principiul MVC pentru a organiza logic codul:

- **Model:** Reprezentat de definițiile Prisma (schema.prisma). Acesta dictează structura datelor și relațiile dintre ele (ex: Un User are mai multe Fișiere).
- **Controller:** Acesta este punctul de intrare pentru cererile HTTP. Controller-ul nu conține logică de business complexă, ci doar preia datele, apelează Serviciul corespunzător și returnează răspunsul (JSON).
- **View:** În contextul unui REST API, "View-ul" este reprezentat de formatul standardizat JSON pe care îl trimitem clientului.

### 2.1.2. API REST (Representational State Transfer)

Comunicarea între client (browser) și server se face printr-un API RESTful.

- **Stateless:** Serverul nu păstrează starea clientului între cereri (folosim JWT pentru autentificare la fiecare request).
- **Resurse:** Totul este privit ca o resursă (ex: /api/users, /api/files).
- **Verbe HTTP:** Folosim corect metodele: GET (citire), POST (creare), PATCH (actualizare parțială), DELETE (ștergere).

### 2.1.3. ORM Code First (Object-Relational Mapping)

Am utilizat **Prisma ORM** cu abordarea **Code First**. Aceasta înseamnă că baza de date este modelată prin cod (fișierul schema.prisma), iar tabelele sunt generate automat pe baza acestui fișier.

- **Avantaj major:** Orice modificare a structurii bazei de date (ex: adăugarea unei coloane noi) este versionată prin Git, permițând echipei să lucreze sincronizat.

### 2.1.4. Service Layer Pattern

Pentru a evita "Fat Controllers" (controllere supraîncărcate), am introdus un strat intermediar numit **Services**.

- **Responsabilitate:** Aici se manageriază toată "inteligența" aplicației. Validările complexe, calculele (ex: verificarea dacă un user mai are spațiu de stocare), interacțiunea directă cu baza de date se fac aici.

## 2.2. Arhitectura Aplicației

Aplicația este împărțită în două mari componente fizice care rulează independent:

1. **Server Backend (Node.js + Express):**
  - Gestionează logica, securitatea și baza de date.
  - Expune documentație prin Swagger UI.
2. **Client Frontend (Next.js + React):**
  - Este o Single Page Application (SPA).
  - Se ocupă exclusiv de randarea interfeței și experiența utilizatorului (UX).

### 2.4. Proiectarea Bazei de Date

Baza de date PostgreSQL este normalizată și conține următoarele entități principale:

1. **User:**
  - id: Primary Key
  - email: Unique
  - password: Hash (bcrypt)

- role: Enum (USER, ADMIN)
- planId: Foreign Key către tabela Plan
- 2. **Plan:**
  - Definește tipurile de abonament (Free, Pro).
  - Conține limitele: maxFiles, maxStorage.
- 3. **File:**
  - Stochează metadatele fișierelor CSV încărcate.
  - content: Conținutul JSON parsat al fișierului.
  - deletedAt: Timestamp pentru Soft Delete.
- 4. **ActivityLog:**
  - Tabelă de audit pentru acțiunile utilizatorilor (ex: "User X a șters fișierul Y").

## Capitolul 3. Implementare

### 3.1. Business Layer (Nivelul Servicii)

Aceasta parte de aplicație este printre cele mai importante care țin corectitudinea datelor pe tot parcursul interacțiunii utilizator și se constituie prin următoarele:

#### 3.1.1. Serviciul de Planuri (PlanService)

Acest serviciu nu doar returnează planurile din baza de date, ci le îmbogățește cu date calculate în timp real.

- **Logica "Is Popular":** Sistemul calculează automat care este cel mai vândut abonament. Dacă un plan are peste X utilizatori activi, primește flag-ul isPopular: true, iar frontend-ul îl va evidenția vizual.
- **Upgrade/Downgrade:** La schimbarea planului, serviciul verifică dacă utilizatorul se încadrează în noile limite. Dacă un user vrea să treacă de la Pro la Free, dar are 50 de fișiere (limita Free fiind 5), sistemul blochează acțiunea și returnează o eroare de business.

#### 3.1.2. Serviciul de Fișiere (FileService)

Gestionează ciclul de viață al fișierelor CSV.

- **Upload:** Fișierul este primit, validat (să fie CSV), parsat rând cu rând și transformat într-un format JSON optimizat pentru stocare în baza de date.
- **Verificare Limite:** Înainte de a salva, se apelează UserContext pentru a vedea dacă utilizatorul mai are dreptul la upload conform abonamentului său.

### 3.1.3. Serviciul de Utilizatori (UserService) - Admin vs User

- **Admin Actions:** Metodele getAllUsers, updateUserPlan și softDeleteUser sunt accesibile doar dacă middleware-ul de securitate confirmă rolul de ADMIN.
- **Securitate date:** Serviciul are grijă să excludă câmpul password din orice răspuns trimis către frontend (DTO Pattern - Data Transfer Object).

### 3.2. Librării Suplimentare Utilizate

Securitatea a fost tratată cu prioritate maximă:

1. Parole: Nu sunt stocate niciodată în clar și se folosește bcrypt.
2. JWT (JSON Web Tokens): La login, serverul emite un token semnat. Clientul trebuie să trimită acest token în header-ul Authorization: Bearer <token> la fiecare cerere.
3. Validare Input: Se folosește librăria Zod pentru a valida orice date vin din formulare (ex: email valid, parolă minim 8 caractere), prevenind SQL Injection sau XSS (cross-site scripting).

### 3.3. Librării suplimentare utilizate

Pentru a eficientiza dezvoltarea, am integrat ecosistemul Node.js:

- Backend:
  - cors: Pentru a permite cereri dinspre Frontend (localhost:3000) către Backend (localhost:4000).
  - morgan: Pentru logarea cererilor HTTP în consolă (debugging).
  - papaparse: Cel mai rapid parser de CSV pentru Node.js.
- Frontend:
  - axios: Client HTTP bazat pe Promise-uri.
  - react-hook-form: Gestiunea formulelor complexe.
  - lucide-react: Set de iconițe moderne și ușoare.

## Capitolul 4. Utilizare

### 4.1. Pașii de Instalare

Instalare și configurare pentru programator

1. Descărcare surse: Se dezarchivează proiectul.
2. Configurare Mediu:
  - În folderul /server, creați .env:

```
DATABASE_URL="postgresql://user:pass@localhost:5432/datainsight"  
JWT_SECRET="supersecretkey"  
PORT=4000
```

### 3. Inițializare Bază de Date:

- Rulați `npm install` pentru a descărca pachetele.
- Rulați `npm run prisma migrate dev --name init` pentru a crea structura bazei de date.
- Rulați `npm run prisma db seed` pentru a popula baza cu date de test (planuri, admin default).
- 4. Pornire server prin comanda `npm run backend$dev`
- 5. Pornire client prin comanda `npm run dev`

## 4.2. Mod de Utilizare

### 4.3.1. Ghid Utilizator

- **Dashboard:** Pagina principală oferă o privire de ansamblu asupra resurselor consumate (spațiu, număr grafice).
- **Încărcare Date:** Butonul "Upload CSV" deschide fereastra de selecție. Fișierul este validat pe loc.
- **Analiză:** După încărcare, utilizatorul poate selecta "Create Chart", alege fișierul sursă și coloanele pe care vrea să le ploteze.

### 4.3.2. Ghid Administrator

- Accesul se face printr-un buton special din meniu, vizibil doar pentru conturile cu rol ADMIN.
- Administratorul are acces la tab-urile: "Users", "Plans", "System Logs".
- **Modificare Plan:** Din lista de utilizatori, adminul poate da click pe "Edit" și schimba planul unui utilizator din Free în Pro manual (ex: dacă plata s-a făcut offline).